

End-to-End Data Analytics Platform

Microsoft Fabric | Public Data | Spain Indicators

A complete data engineering portfolio project covering ingestion, transformation, warehousing, and reporting of Spanish public data sources using the Microsoft Fabric ecosystem.

AUTHOR

Analytics Engineer

6 years experience

Microsoft Stack

STACK

Microsoft Fabric

Lakehouse | Warehouse

PySpark | Power BI

DATE

May 2026

Duration: 1-2 months

Version 1.1

1. Executive Summary

This document defines the scope, objectives, and strategic decisions for a complete end-to-end data analytics project built on Microsoft Fabric. The project ingests, transforms, models, and visualizes public indicator data from Spain using five diverse data sources across multiple formats (XLS, XLSX, CSV, REST API, and SQL Server).

The primary goal is twofold: to apply the Microsoft Fabric platform end-to-end on a non-trivial real-world dataset, beyond certification theory (DP-600), and to build a professional portfolio that demonstrates real-world data engineering capabilities for the freelance market in Europe and the United States.

Every component of this project is designed and documented as if it were a client deliverable: clean code, justified technical decisions, professional documentation, and a well-structured GitHub repository.

2. Professional Profile

Role	Analytics Engineer (transitioning from Senior Data Analyst)
Experience	6 years in data analytics across consulting firms (Argentina & Spain)
Location	Barcelona, Spain (Full Remote)
Certifications	PL-300 (Power BI Data Analyst) DP-600 (Fabric Analytics Engineer)
Core Stack	Power BI, SQL Server, SSIS, PowerShell, Azure Data Factory, Microsoft Fabric
Target	Freelance Analytics Engineer (Remote EU/USA)

Relevant Background

For the past two years, the author has led a project for a public entity in Spain, ingesting and normalizing 80+ public data sources (CSV, legacy Excel, XLSX, REST API, web scraping) into a SQL Server data warehouse with a star schema model (fact table, dimension tables for indicators, autonomous communities, and calendar). The solution was orchestrated with SSIS in a fully on-premise environment and delivered through Power BI dashboards and web applications.

This portfolio project takes that accumulated domain knowledge and re-implements a curated subset of the solution using Microsoft Fabric's modern cloud-native stack, demonstrating the ability to migrate and modernize on-premise data platforms.

3. Project Scope

What This Project Covers

- Ingestion of 5 carefully selected public data sources covering different formats and access patterns.
- Bronze-Silver-Gold medallion architecture implemented in Microsoft Fabric Lakehouse.
- Data transformation using PySpark Notebooks and Dataflows Gen2, each applied where architecturally appropriate.
- Star schema dimensional model in Fabric Warehouse for analytical querying.
- Power BI report integrated within the Fabric ecosystem (Direct Lake connectivity).
- End-to-end orchestration using Data Pipelines.
- Complete professional documentation (functional and technical) delivered as portfolio-ready artifacts.

What This Project Does NOT Cover

- Production deployment or real-time data refresh schedules.
- User access management, RLS, or multi-tenant configurations.
- The full 80+ source dataset from the original client project.
- Advanced data science or machine learning workloads.

4. Technology Stack & Key Decisions

Component	Technology	Justification
Storage	OneLake (Lakehouse)	Unified storage layer, Delta Lake format, medallion architecture
Ingestion	Data Pipelines	Orchestration of diverse sources with scheduling capabilities
Transformation	Notebooks (PySpark)	Complex transformations, data cleansing, schema enforcement
Light ETL	Dataflows Gen2	Simple transformations where Spark overhead is unnecessary
Data Warehouse	Fabric Warehouse	Star schema model, SQL-based analytical layer
Reporting	Power BI (Direct Lake)	Native Fabric integration, optimal performance
Version Control	GitHub	Code, documentation, and portfolio presentation

Key Architectural Principle: Each Fabric component is used where it naturally fits. Notebooks handle complex PySpark transformations, Dataflows Gen2 handle simple mappings, and Pipelines orchestrate the flow. Nothing is forced to demonstrate tool usage; instead, each choice is justified by the nature of the data and the transformation required.

5. Licensing & Identity Strategy

The project uses a fully independent personal account stack rather than employer-issued infrastructure. This decision is foundational and affects every downstream technical choice (authentication, capacity, deployment).

5.1 Why Personal Account (not Corporate)

- **Professional independence:** the portfolio is the author's personal property, not bound to any employer's IT policies or governance.
- **Live demo capability:** the workspace can be screen-shared with potential freelance clients without exposing employer infrastructure or breaching internal policies.
- **Portfolio continuity:** if employment changes in the coming months, the portfolio remains fully accessible and demonstrable.
- **Zero policy conflict:** avoids any ambiguity around side-project ownership or the use of corporate identities for personal work.

5.2 Dual-Identity Setup (Personal Entra ID Tenant)

Microsoft Fabric does not accept Microsoft Personal Accounts (MSA) directly: it requires a work-or-school identity inside an Entra ID tenant. The solution is a personal Entra ID tenant where the author owns both identities:

Role	Identity	Used For
Tenant owner (Azure)	gaston.baloira@outlook.com (Microsoft Personal Account)	Owning the personal Azure subscription and tenant administration
Fabric / SQL technical user	gaston@gastonbaloiraoutlook.onmicrosoft.com (Entra ID work-or-school)	Logging into Fabric, owning workspaces, connecting to Azure SQL via Entra-only auth

Both identities live inside the same personal tenant `gastonbaloiraoutlook.onmicrosoft.com`, so there are no cross-tenant complications. The Entra ID Free license is sufficient for all the project's needs.

5.3 Fabric Capacity Strategy

The capacity strategy is split into two phases, designed to maximize value during development and preserve a live portfolio after development ends.

Phase	Capacity	Cost	Purpose
Development (Phase 5)	Fabric Trial (F64 equivalent, 60 days)	Free	Active development, full feature access, fast iteration
Post-development (portfolio life)	F2 PAYG (Pay-As-You-Go, paused by default)	\$0 paused ~\$0.36/h active	On-demand demo capability for freelance client interviews; activated only for live screen-shares

Religious pause practice: the F2 PAYG capacity is started only for the duration of a demo or active iteration session, then paused immediately. Estimated typical monthly cost under this discipline: \$5–15 USD. This simulates the cost-controlled consumption pattern of a real client engagement.

5.4 Capacity Management Timeline

- **Phases 0–4 (planning):** completed before activating any capacity, ensuring no trial days are wasted on documentation work.
- **Trial activation (Day 1 of Phase 5):** triggered when development is ready to begin, providing the full 60 days for hands-on Fabric work.
- **Trial extension:** extendable for an additional 60 days when fewer than 7 days remain (Microsoft policy).
- **Migration to F2 PAYG:** performed before day 53 of the trial to ensure continuity, with the workspace already migrated and tested before the trial expires.
- **Post-trial preservation:** all code, documentation, screenshots, and architecture diagrams are stored in GitHub, ensuring the portfolio persists independently of the Fabric capacity state.

6. Project Phases & Deliverables

- 0

Context & Strategic Decisions
Deliverable: Project Brief (PDF)

COMPLETE
- 1

Requirements & Source Selection
Deliverable: Requirements Document + Source Catalog

NEXT
- 2

Architecture Design
Deliverable: Architecture Diagram + Design Document

PENDING
- 3

Functional Specification
Deliverable: Functional Document (PDF)

PENDING
- 4

Technical Specification
Deliverable: Technical Document (PDF)

PENDING
- 4.5

Local Python Validation
Deliverable: Local scripts validating transformation logic with real data

PENDING
- 5

Development in Fabric
Deliverable: Working Solution + Screenshots + Code

PENDING
- 6

GitHub Publication
Deliverable: Complete Repository + Professional README

PENDING

6.1 Phase 4.5 — Local Python Validation

An intermediate validation phase has been inserted between Phase 4 (Technical Specification) and Phase 5 (Fabric Development). Its purpose is to validate transformation logic with real data using Python locally before moving to PySpark notebooks in Fabric.

Rationale

- **De-risk the trial window:** debugging transformation logic against real data outside Fabric saves precious trial-window hours.
- **Pre-validated logic:** when entering Fabric, the transformation logic is already proven correct — the Phase 5 work focuses on the Fabric-specific parts (Delta writes, Spark execution, pipeline orchestration), not on rediscovering source quirks.
- **First deliverable:** the IPI source has already been loaded to Azure SQL via `ipi_bronze_load.py` (309K rows, ~30 seconds with bulk insert).
- **Discovery surface:** implementation patterns surfaced during this phase (Azure SQL connection, Entra-only auth, ODBC ARM64 setup) are documented in Phase 4 as portfolio-grade content.

Estimated Timeline

Phase	Duration	Fabric Required
Phases 0–4 (Planning & Design)	2–3 weeks	No
Phase 4.5 (Local Python Validation)	1 week	No
Phase 5 (Development)	3–4 weeks	Yes (Trial active)
Phase 6 (Publication)	1 week	No (or F2 PAYG paused)

7. Documentation Strategy

All project documentation follows a multi-format approach optimized for both GitHub browsability and professional client delivery:

Format	Purpose	Location
README.md	Landing page: project overview, architecture, quick navigation	Repository root
PDF Documents	Formal deliverables: project brief, functional spec, technical spec	/docs
Inline Comments	Code documentation in notebooks and scripts	Within code files
Curated Screenshots	8–15 high-impact captures of Fabric workspace, pipelines, and report (quality over quantity)	/docs/screenshots
Demo Video (Loom)	3–5 minute screen-share showing the solution end-to-end (added Phase 5)	/docs/demo
Architecture Diagrams	Visual system design and data flow documentation	/docs/diagrams
Lessons Learned	Living markdown for findings during Phase 5 (not re-versioning the formal PDFs)	/docs/lessons-learned.md

Curated Screenshots vs. Massive Captures

An earlier draft of this strategy assumed dozens of screenshots covering every step of the build. This was reconsidered for two reasons: (1) reviewers and freelance clients do not read 50 screenshots; (2) the F2 PAYG capacity strategy enables on-demand live demos, which are more compelling than any static capture. The new approach prioritizes 8–15 carefully chosen screenshots that tell the story end-to-end, plus a short Loom video as the headline demo asset.

8. Quality Standards

This project adheres to professional standards as if it were a real client engagement:

- **Code Quality:** Clean, well-commented code following consistent naming conventions across all Fabric components.
- **Decision Transparency:** Every architectural and technical decision is documented with its rationale, including why alternatives were not chosen.
- **Reproducibility:** While the Fabric workspace can be paused or migrated, all code, configurations, and visual evidence are preserved in the repository.
- **Professional Presentation:** Documentation quality matches what would be delivered to a paying client, reinforcing the freelance-ready positioning of this portfolio.
- **Live Demonstrability:** the F2 PAYG capacity allows reactivation of the workspace on demand for client interviews, going beyond a purely static portfolio.

9. Repository Structure

The GitHub repository is organized to clearly separate formal deliverables, one-time setup, and productive Phase 5 code. This structure replaces the simpler structure planned in v1.0.

```

spi-spain-indicators/
  ■■■■ README.md                (project landing page)
  ■■■■ LICENSE                  (MIT)
  ■■■■ .gitignore
  ■■■■ requirements.txt         (global project dependencies)
  ■
  ■■■■ docs/                    (formal documentation)
  ■   ■■■■ phase0_project_brief.pdf
  ■   ■■■■ phase1_requirements.pdf
  ■   ■■■■ phase2_architecture.pdf
  ■   ■■■■ phase3_functional.pdf
  ■   ■■■■ phase4_technical.pdf
  ■   ■■■■ diagrams/            (architecture diagrams PNG/SVG)
  ■   ■■■■ screenshots/         (curated Fabric captures - Phase 5)
  ■   ■■■■ demo/                (Loom video link / embed)
  ■   ■■■■ lessons-learned.md   (living markdown for findings)
  ■
  ■■■■ prerequisites/           (one-time setup scripts)
  ■   ■■■■ README.md            (what this folder is for)
  ■   ■■■■ ipi-azure-sql-load/
  ■       ■■■■ README.md
  ■       ■■■■ ipi_bronze_load.py
  ■       ■■■■ requirements.txt  (component-specific deps)
  ■
  ■■■■ src/                     (productive pipeline code - Phase 5)
  ■   ■■■■ notebooks/           (PySpark/Python Fabric notebooks)
  ■       ■■■■ README.md
  ■       ■■■■ bronze/
  ■       ■■■■ silver/
  ■       ■■■■ gold/
  ■   ■■■■ pipelines/           (Data Pipelines JSON definitions)
  ■       ■■■■ README.md
  ■   ■■■■ dataflows/           (Dataflows Gen2 definitions)
  ■       ■■■■ README.md
  ■   ■■■■ warehouse/           (Warehouse DDL scripts)
  ■       ■■■■ README.md
  ■   ■■■■ power-bi/            (PBIP files)
  ■       ■■■■ README.md

```

Key Principles

- docs/ = formal deliverables (frozen PDFs) + living markdown for ongoing findings.
- prerequisites/ = one-time setup scripts, not part of the productive pipeline.
- src/ = productive Phase 5 code (notebooks, pipelines, warehouse, PBIP).
- Source files (CSVs, XLS) are NOT versioned — too large for Git; documented for download instead.
- .venv/, .DS_Store, and any private folders are gitignored — local-only, not for portfolio.

10. Changelog

This document is now considered a frozen deliverable. Future findings during Phase 5 (Fabric development) will be captured in the GitHub repository (README, lessons-learned.md, and per-component READMEs), not by re-versioning this PDF.

Version 1.1 — May 2026

Strategic refresh applied before Phase 5 kickoff. Five substantive changes; no impact on the technical architecture (medallion pattern, star schema, source-to-target mappings, transformation logic remain unchanged).

Area	Change	Section
Author role	Updated from 'Senior Data Analyst' to 'Analytics Engineer' to reflect the target professional positioning	Cover, Section 2
Account strategy	Switched from corporate (employer-issued) to personal account stack for full professional independence and live-demo capability	Section 5.1, 5.2
Capacity strategy	Extended from Trial-only to Trial + F2 PAYG (paused by default). Enables a permanently demonstrable portfolio at minimal cost	Section 5.3, 5.4
Documentation strategy	Pivoted from massive screenshot capture to curated 8–15 screenshots + Loom demo video for higher-impact storytelling	Section 7
Phase 4.5 added	New intermediate phase: Local Python Validation between Phase 4 and Phase 5. Validates transformation logic with real data before Fabric work begins	Section 6
Repository structure	Refined to explicitly separate formal documentation, one-time setup (prerequisites), and productive code (src). Adds dedicated locations for diagrams, screenshots, and lessons learned	Section 9

Version 1.0 — March 2026

Initial Phase 0 deliverable. Defined project scope, professional profile, technology stack, phases, documentation strategy, quality standards, and repository structure.

This document serves as the foundational reference for the project. All subsequent phases build upon the scope, decisions, and standards defined here.

Document version 1.1 | May 2026 | Phase 0 deliverable (frozen)